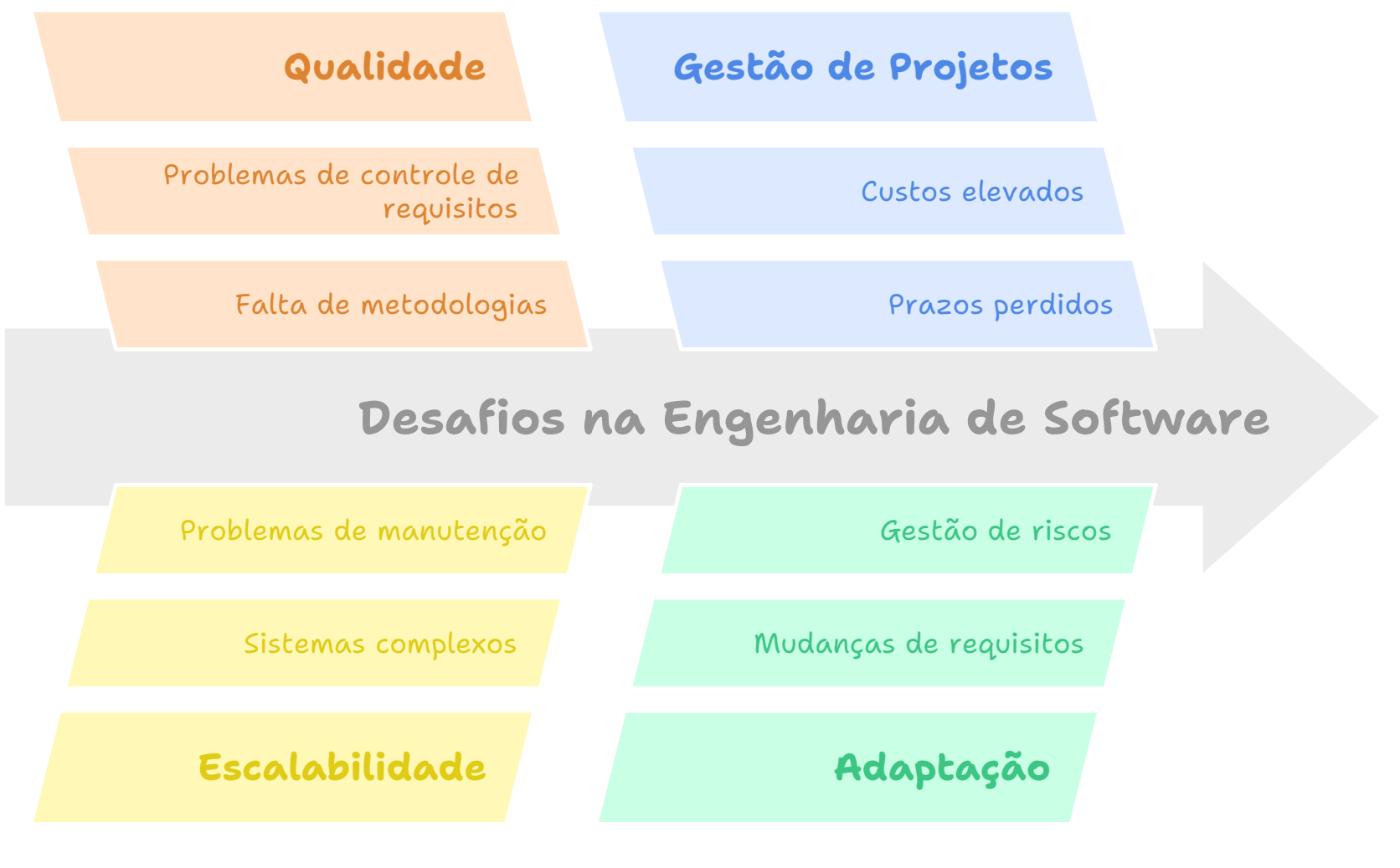
AULA 2 – HISTÓRICO DA ENGENHARIA DE SOFTWARE E CICLO DE VIDA DO SOFTWARE

1. Título – Histórico da Engenharia de Software

- **Definição:**Engenharia de Software é a aplicação de princípios de engenharia para o desenvolvimento, operação e manutenção de software.

- **Origem:**
  - Década de 1960 → Aumento da complexidade dos sistemas computacionais.
  - A crise do software revelou a necessidade de metodologias estruturadas.
- **Primeiros desafios:**
  - Falta de qualidade nos sistemas desenvolvidos.
  - Problemas de escalabilidade e manutenção.
  - Custos elevados e prazos não cumpridos.
- **Importância:**
  - Melhorar a previsibilidade dos projetos.
  - Garantir qualidade e escalabilidade.
  - Manter o controle sobre os requisitos e mudanças.

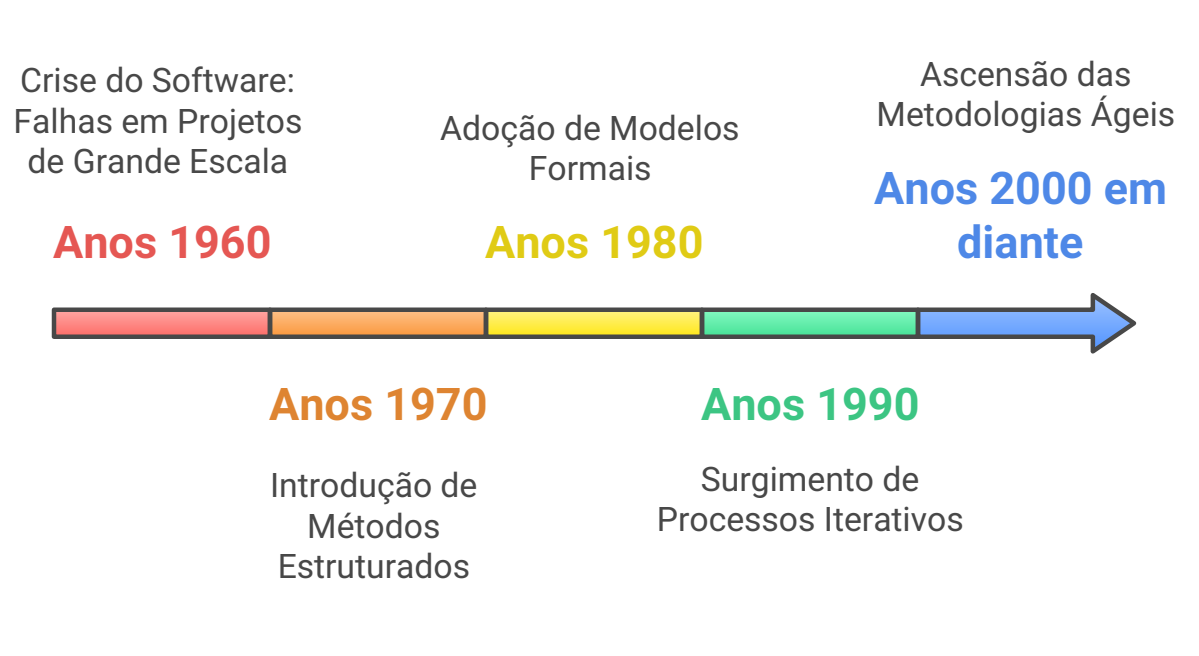
Desafios na Engenharia de Software


Qualidade
Problemas de controle de requisitos
Falta de metodologias
Gestão de Projetos
Custos elevados
Prazos perdidos
Desafios na Engenharia de Software
Problemas de manutenção
Sistemas complexos
Escalabilidade
Gestão de riscos
Mudanças de requisitos
Adaptação

2. Fases da Evolução da Engenharia de Software

- **Década de 1960** – Crise do Software → Falha em projetos de grande escala.
- **Década de 1970** – Métodos estruturados → Desenvolvimento sequencial e
- **Década de 1980** – Modelos formais → Introdução de métricas de qualidade.
- **Década de 1990** – Processos iterativos → Desenvolvimento cíclico e adaptável.
- **Década de 2000 em diante** – Metodologias ágeis → Foco em entregas rápidas e adaptativas.

Evolução da Engenharia de Software: Principais Marcos


Crise do Software: Falhas em Projetos de Grande Escala
Anos 1960
Adoção de Modelos Formais
Anos 1980
Ascensão das Metodologias Ágeis
Anos 2000 em diante
Anos 1970
Introdução de Métodos Estruturados
Anos 1990
Surgimento de Processos Iterativos

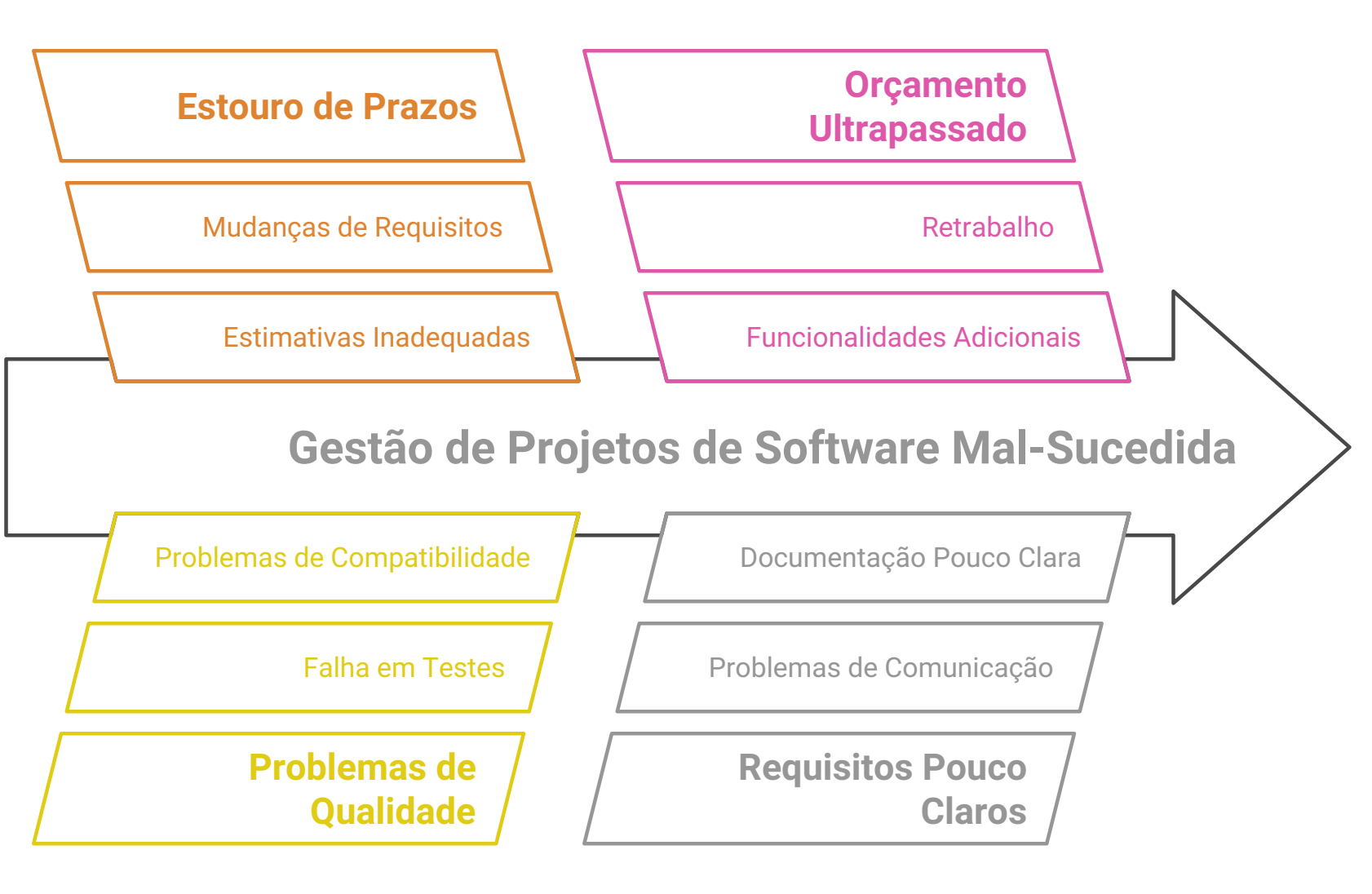
Exemplo:

- Crise do software – Sistema de controle aéreo com falhas em tempo real → Necessidade de processos formais e testes rigorosos.

3. Problemas enfrentados na engenharia de software

- **Estouro de prazos:**
  - Estimativas inadequadas.
  - Falta de recursos ou mudança nos requisitos.
- **Problemas de qualidade:**
  - Falha em testes.
  - Problemas de compatibilidade.
- **Orçamento ultrapassado:**
  - Retrabalho por requisitos mal definidos.
  - Adição de novas funcionalidades sem planejamento.
- **Falta de requisitos claros:**
  - Dificuldade na comunicação com stakeholders.
  - Falta de clareza na documentação.

Desafios na Gestão de Projetos de Software


Estouro de Prazos
Mudanças de Requisitos
Estimativas Inadequadas
Orçamento Ultrapassado
Retrabalho
Funcionalidades Adicionais
Gestão de Projetos de Software Mal-Sucedida
Problemas de Compatibilidade
Falha em Testes
Problemas de Qualidade
Documentação Pouco Clara
Problemas de Comunicação
Requisitos Pouco Claros

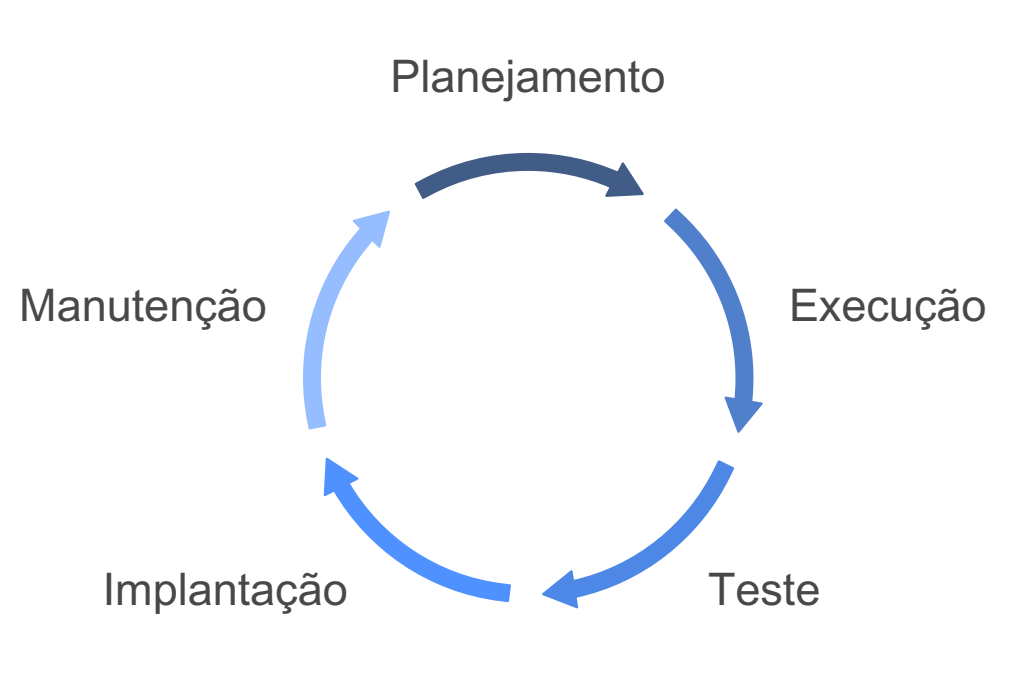
Exemplo:

- Projeto do Healthcare.gov → Problemas de estabilidade e segurança devido a requisitos vagos e mudanças tardias.

4. Ciclo de Vida de Software – Definição

- **Conceito:**
  - Conjunto de fases que um software atravessa, desde o planejamento até a manutenção.
- **Importância:**
  - Redução de riscos.
  - Maior controle sobre o escopo.
  - Melhor capacidade de adaptação a mudanças.

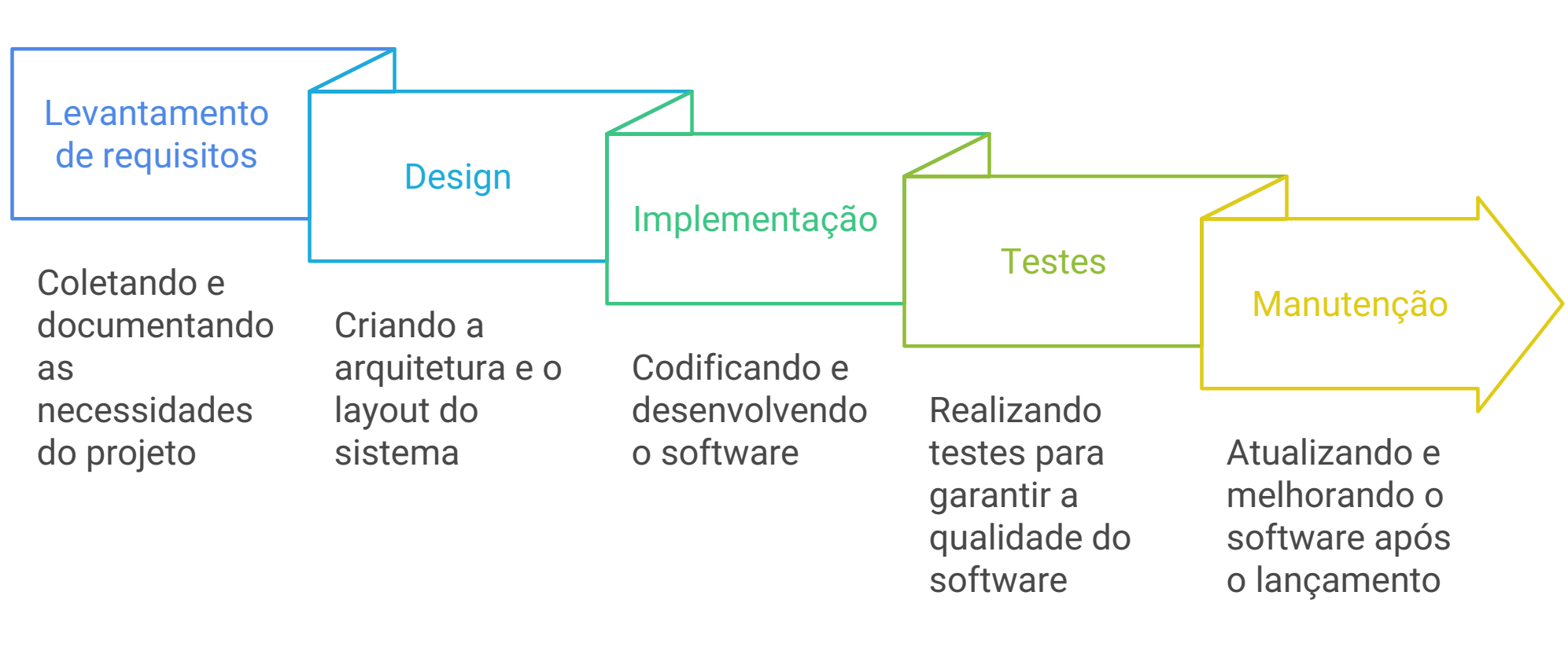
Ciclo de Vida do Software


Planejamento
Execução
Teste
Implantação
Manutenção

5. Modelo Cascata – Definição e Etapas

- **Etapas:**
  - Levantamento de requisitos.
  - Design.
  - Implementação.
  - Testes.
  - Manutenção.
- **Vantagens:**
  - Processo estruturado e fácil de controlar.
  - Boa documentação.
- **Desvantagens:**
  - Rigidez → Dificuldade em lidar com mudanças.
  - Testes tardios → Problemas detectados apenas ao final.

Etapas do Modelo de Ciclo de Vida de Software Cascata


Levantamento de requisitos
Coletando e documentando as necessidades do projeto
Design
Criando a arquitetura e o layout do sistema
Implementação
Codificando e desenvolvendo o software
Testes
Realizando testes para garantir a qualidade do software
Manutenção
Atualizando e melhorando o software após o lançamento

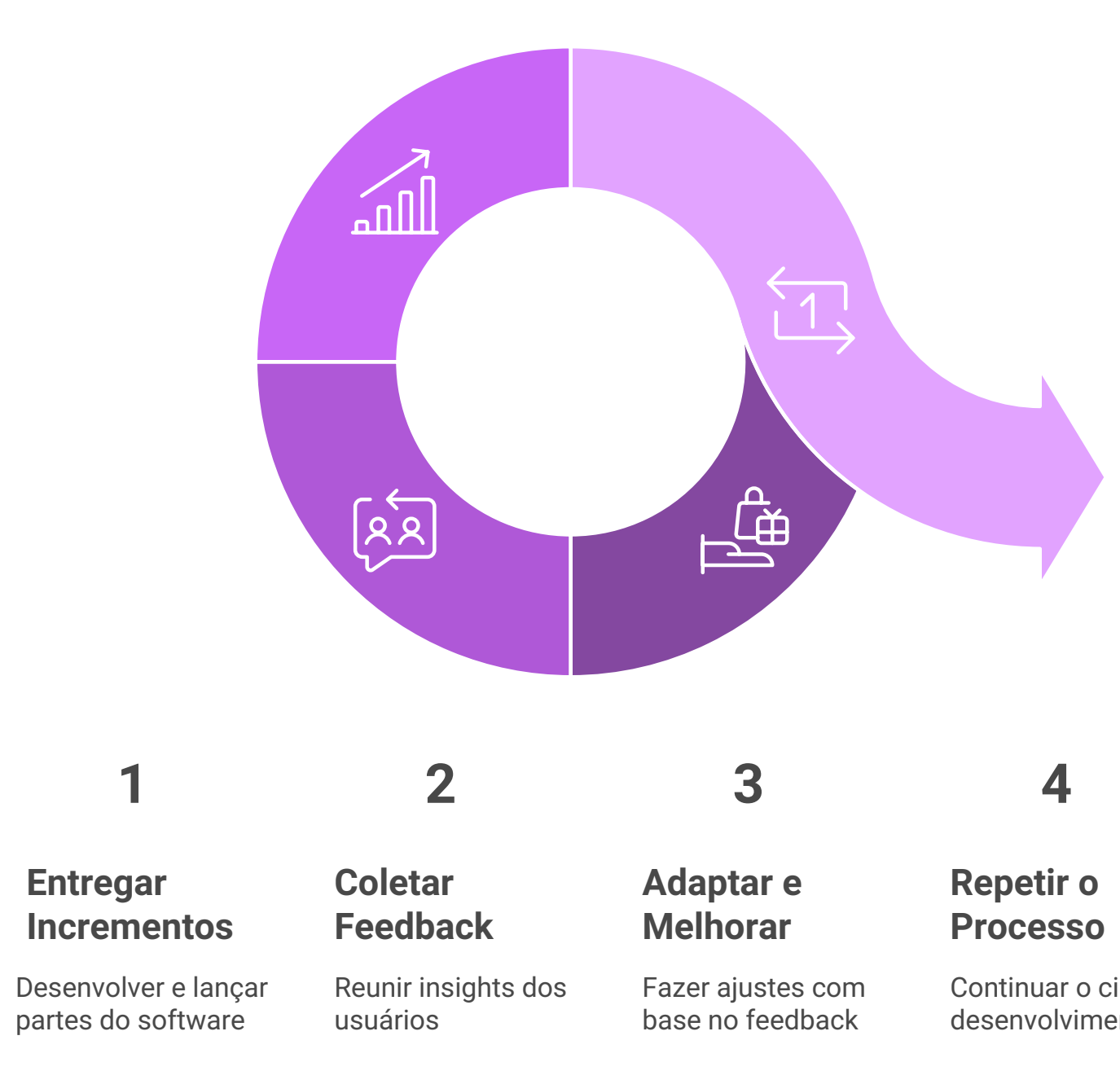
Exemplo:

- Desenvolvimento de sistemas bancários → Segue o modelo cascata por necessidade de rigor e controle.

6. Modelo Iterativo – Definição e Características

- **Conceito:**
  - Desenvolvimento incremental, com entregas parciais.
- **Características:**
  - Feedback constante.
  - Melhor adaptação a mudanças.
- **Exemplo:**
  - Desenvolvimento de um aplicativo de redes sociais → Feedback contínuo de usuários para ajustes rápidos.

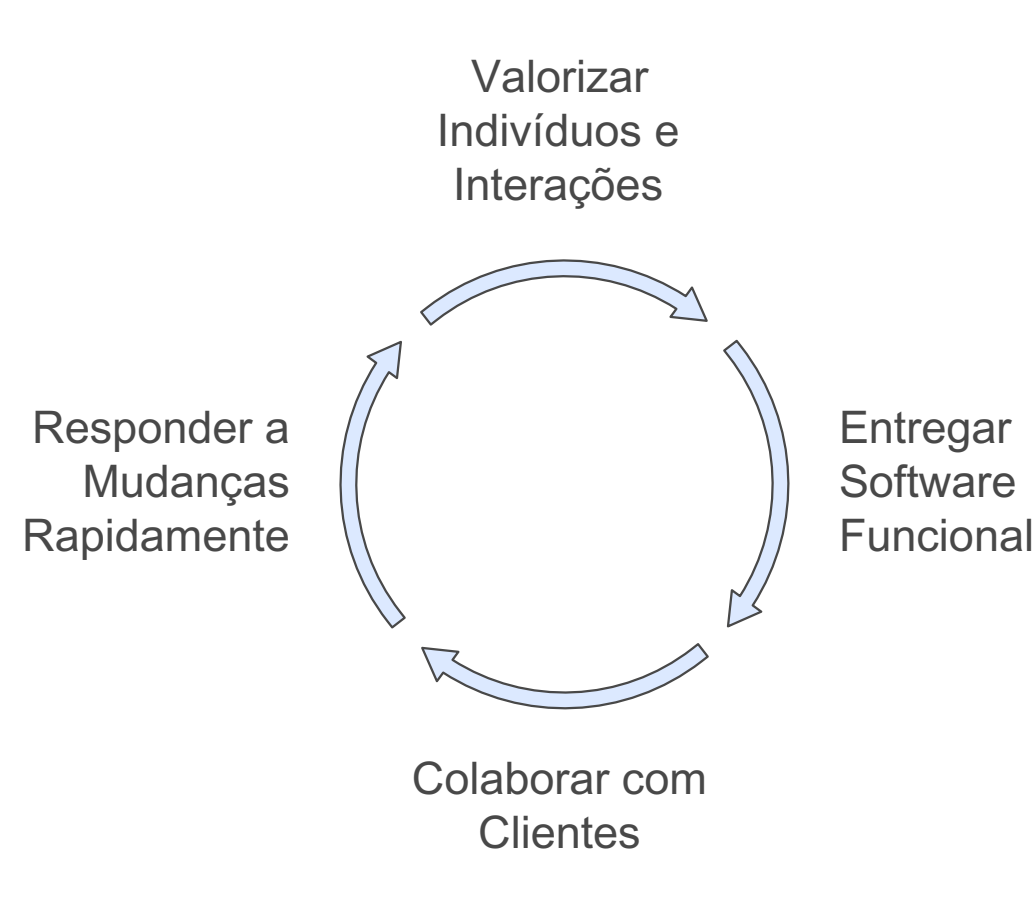
Ciclo de Desenvolvimento de Software Ágil


1
Entregar Incrementos
Desenvolver e lançar partes do software
2
Coletar Feedback
Reunir insights dos usuários
3
Adaptar e Melhorar
Fazer ajustes com base no feedback
4
Repetir o Processo
Continuar o ciclo de desenvolvimento

7. Modelo Ágil – Definição e Princípios

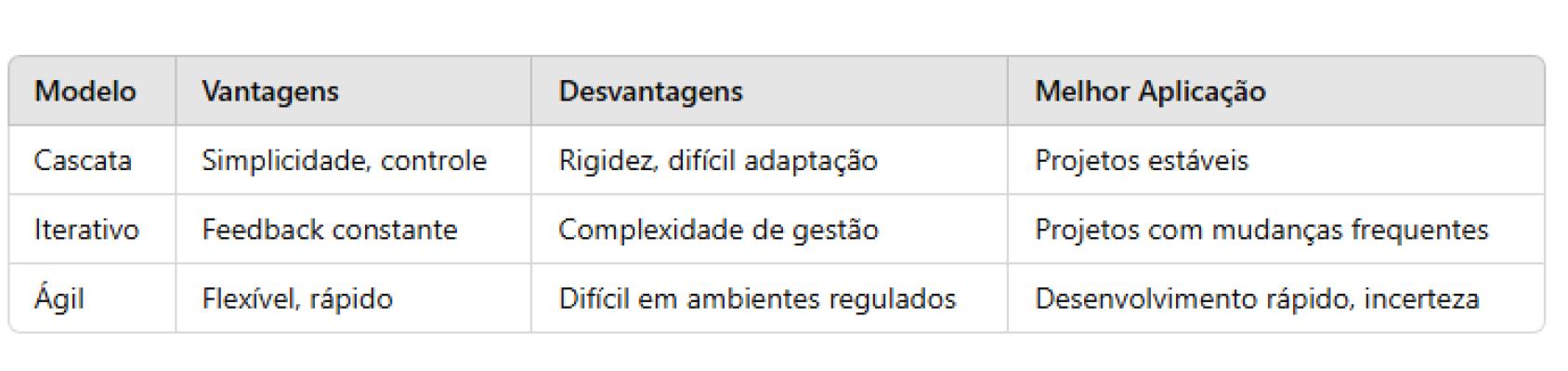
- **Conceito:**
  - Desenvolvimento incremental e adaptativo.
- **Princípios:**
  - Indivíduos e interações acima de processos e ferramentas.
  - Software funcional acima de documentação extensa.
  - Colaboração com o cliente acima de negociação de contrato.
  - Resposta rápida a mudanças acima de seguir um plano.

Ciclo de Desenvolvimento Ágil


Valorizar Indivíduos e Interações
Entregar Software Funcional
Colaborar com Clientes
Responder a Mudanças Rapidamente

8. Comparação entre os modelos

ModeloVantagensDesvantagensMelhor Aplicação
Cascata | Simplicidade, controleRigidez, difícil adaptação | Projetos estáveis
Iterativo | Feedback constanteComplexidade de gestão | Projetos com mudanças frequentes



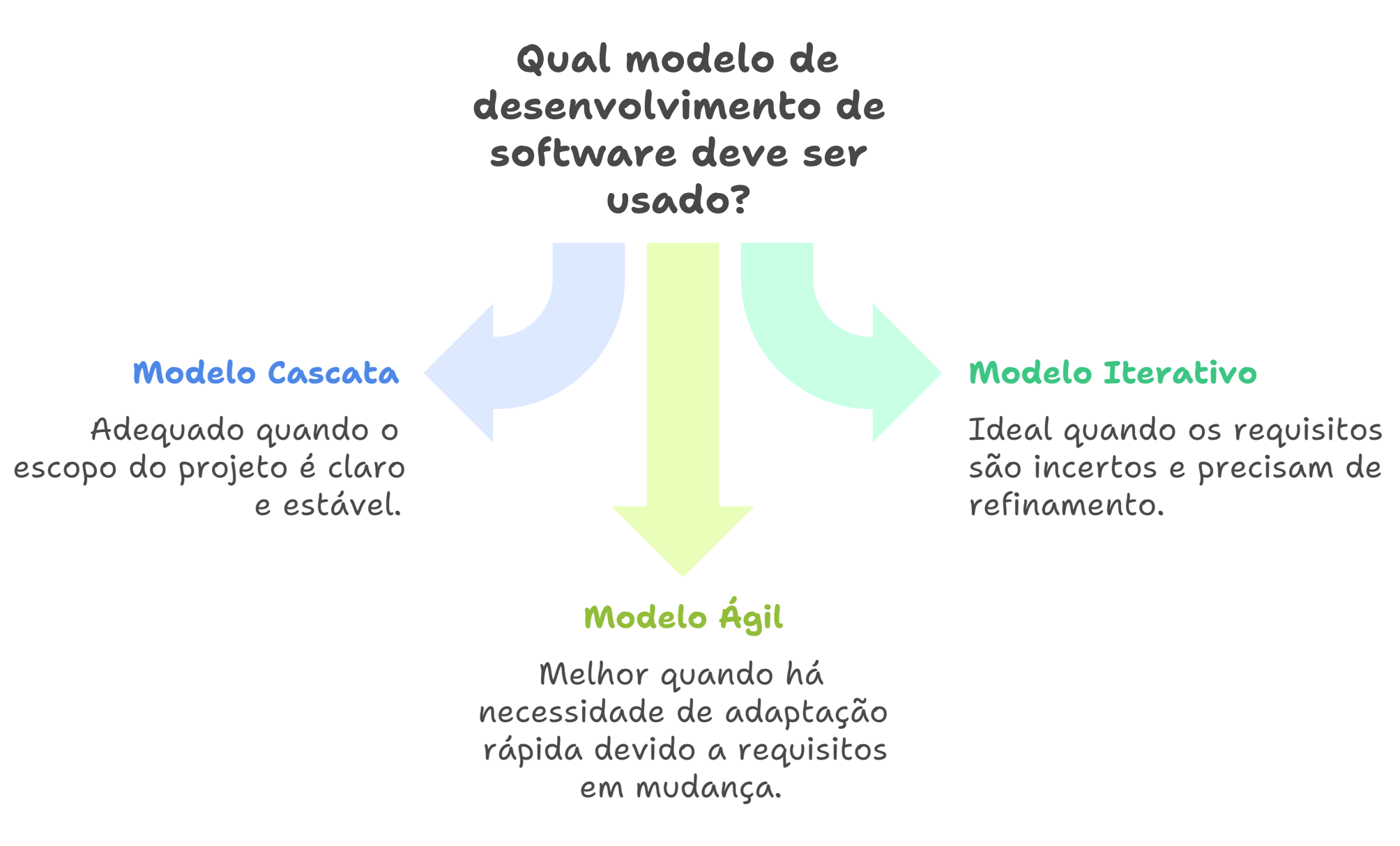
Modelo	Vantagens	Desvantagens	Melhor Aplicação
Cascata	Simplicidade, controle	Rigidez, difícil adaptação	Projetos estáveis
Iterativo	Feedback constante	Complexidade de gestão	Projetos com mudanças frequentes
Ágil	Flexível, rápido	Difícil em ambientes regulados	Desenvolvimento rápido, incerteza

Exemplo:

- Cascata → Software para equipamentos médicos.
- Iterativo → Aplicação de análise de dados.
- Ágil → Aplicativo de e-commerce.

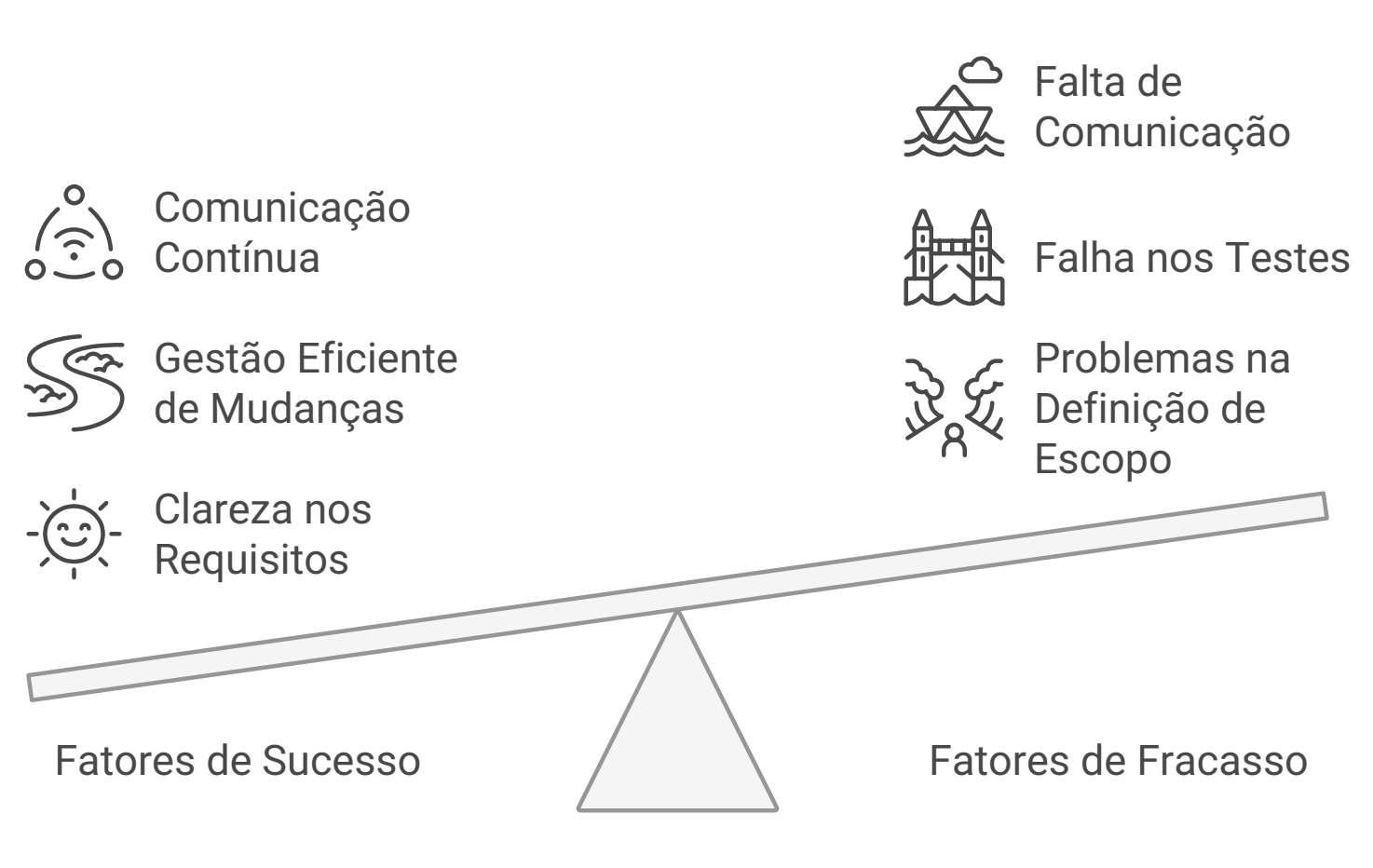
9. Quando utilizar cada modelo?

- **Cascata:** Quando o escopo é claro e estável.
- **Iterativo:** Quando há incerteza nos requisitos.
- **Ágil:** Quando há mudanças constantes e necessidade de rápida adaptação.


Qual modelo de desenvolvimento de software deve ser usado?
Modelo Cascata
Adequado quando o escopo do projeto é claro e estável.
Modelo Iterativo
Ideal quando os requisitos são incertos e precisam de refinamento.
Modelo Ágil
Melhor quando há necessidade de adaptação rápida devido a requisitos em mudança.

10. Análise de sucesso e fracasso – Definição de fatores críticos

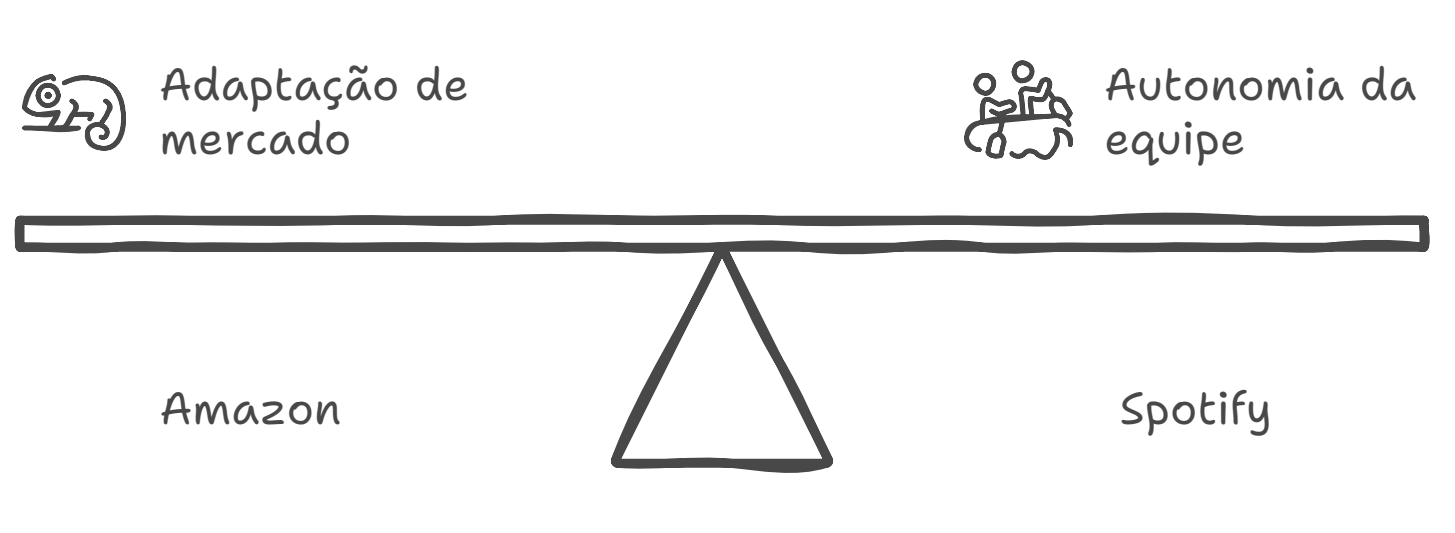
- **Sucesso:**
  - Clareza nos requisitos.
  - Gestão eficiente de mudanças.
  - Comunicação contínua.
- **Fracasso:**
  - Problemas na definição de escopo.
  - Falha nos testes.


Comunicação Contínua
Gestão Eficiente de Mudanças
Clareza nos Requisitos
Falta de Comunicação
Falha nos Testes
Problemas na Definição de Escopo
Fatores de Sucesso
Fatores de Fracasso

Equilibrando Fatores de Sucesso e Fracasso em Projetos de Software

11. Exemplos de sucesso

- **Amazon:**
  - Desenvolvimento iterativo e ágil → Melhor adaptação às demandas do mercado.
- **Spotify:**
  - Modelo ágil baseado em squads → Equipes autônomas e entregas rápidas.


Adaptação de mercado
Autonomia da equipe
Amazon
Spotify

Comparando Agilidade e Autonomia em Modelos de Software

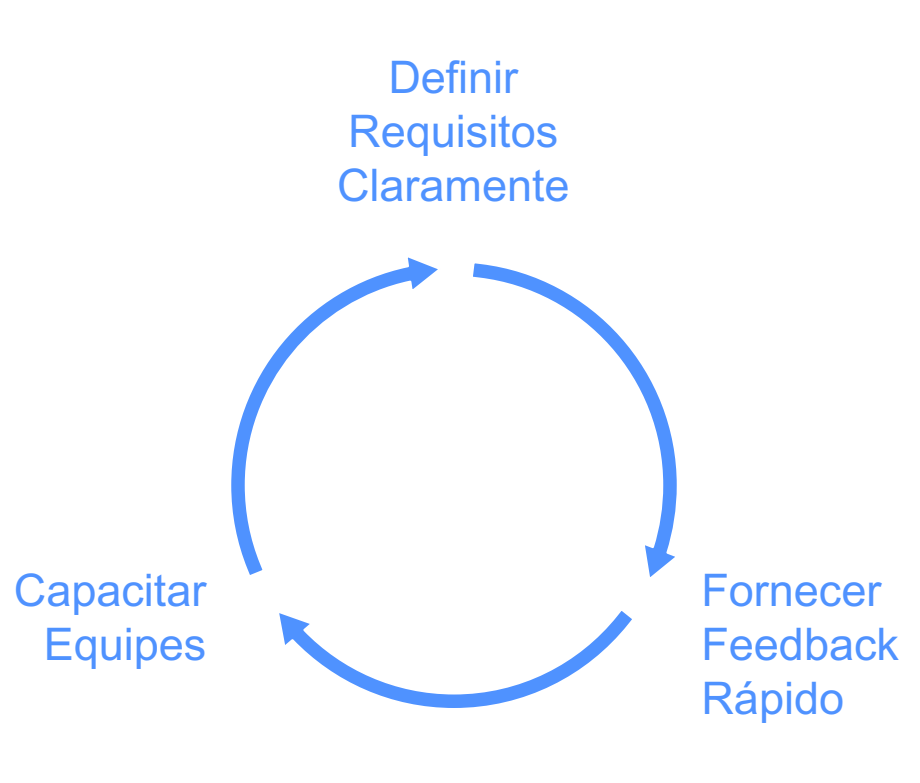
12. Exemplos de fracasso

- **Healthcare.gov:** Problemas técnicos devido à falta de testes e requisitos mal definidos.
- **Windows Vista:** Atrasos e problemas de compatibilidade devido à falta de clareza nos requisitos.

13. Fatores que levaram ao sucesso

- Definição clara de requisitos.
- Feedback rápido e adaptação contínua.
- Equipes capacitadas e engajadas.

Fatores Chave para o Sucesso do Projeto de Software


Definir Requisitos Claramente
Fornecer Feedback Rápido
Capacitar Equipes

14. Fatores que levaram ao fracasso

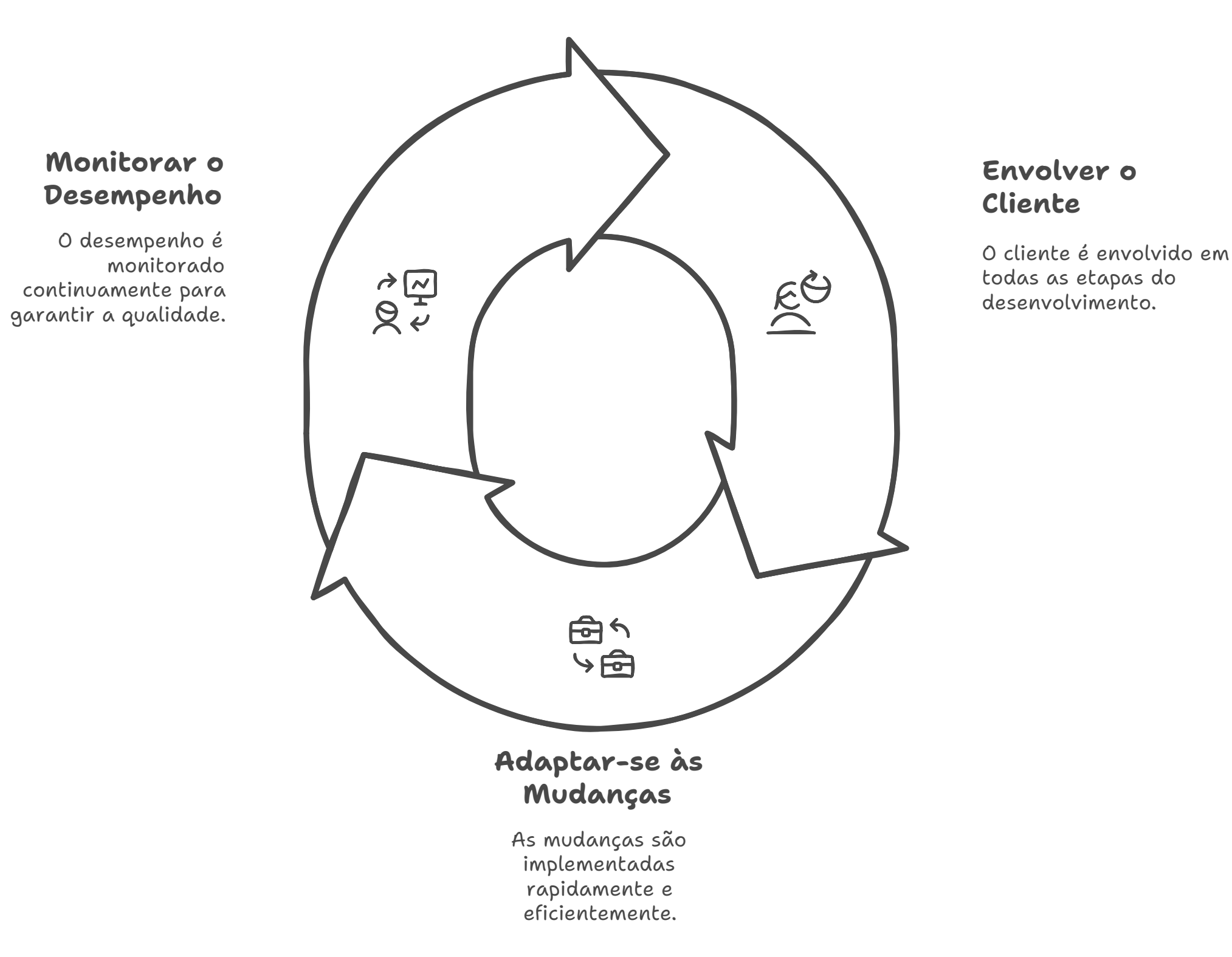
- Requisitos vagos.
- Mudanças tardias e sem controle.
- Testes insuficientes.

Requisitos Vagos
Mudanças Tardias
Testes Insuficientes

15. Lições aprendidas

- Envolver o cliente em todas as etapas.
- Adaptação rápida às mudanças.
- Monitoramento contínuo do desempenho.

Lições aprendidas


Monitorar o Desempenho
O desempenho é monitorado continuamente para garantir a qualidade.
Envolver o Cliente
O cliente é envolvido em todas as etapas do desenvolvimento.
Adaptar-se às Mudanças
As mudanças são implementadas rapidamente e eficientemente.

16. Importância da escolha do modelo correto

- Afeta diretamente a qualidade e eficiência do projeto.
- Reduz custos e riscos.
- Melhora a previsibilidade e controle.

17. Ferramentas para gestão de ciclo de vida

- **Jira:** Backlog, sprints e métricas.
- **Trello:** Gestão visual de tarefas.
- **Azure DevOps:** Integração com ciclo de vida completo.

18. Boas práticas na implementação de modelos

- Comunicação constante entre equipes.
- Revisão contínua dos requisitos.
- Monitoramento e métricas claras.

19. Papel do Product Owner no ciclo de vida

- Definição de backlog.
- Priorização de tarefas.
- Comunicação com stakeholders.

20. Conclusão – Ciclo de vida e sucesso do projeto

- Modelo correto → Maior eficiência e qualidade.
- Feedback contínuo → Melhor adaptação a mudanças.
- Monitoramento → Previsibilidade e controle